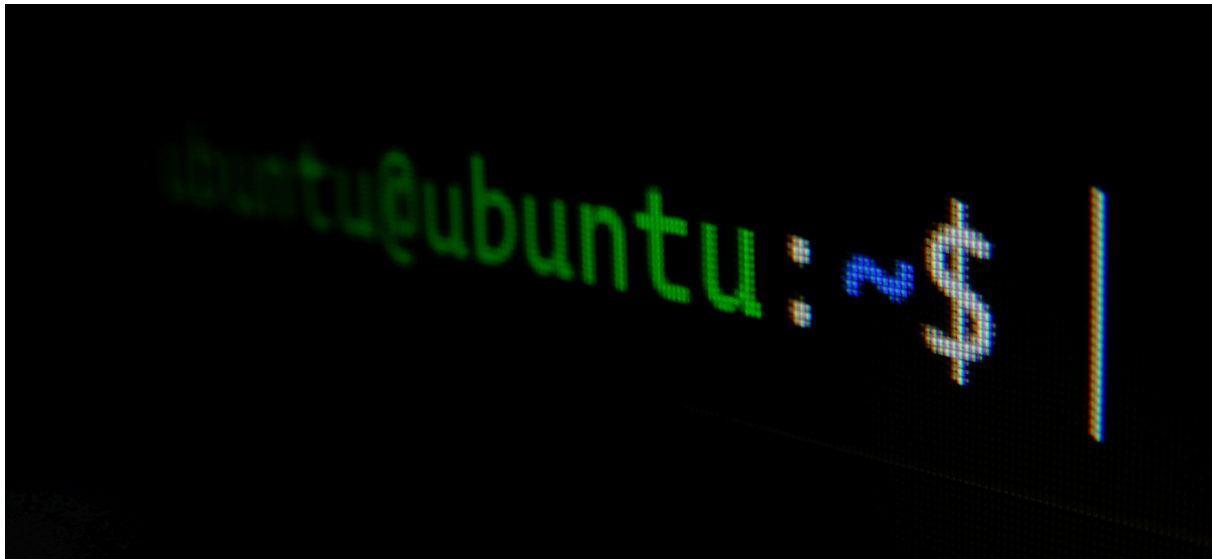




CHAPTER 12 - MANAGING THE LINUX KERNEL

Every operating system consists of **two primary parts**. The core component is the **kernel**, which sits at the heart of the system and oversees all operations - from memory allocation and CPU management to display output. The second part, commonly called **user space**, encompasses virtually all other system elements.

The kernel operates as a secure, restricted zone accessible only to administrators and other high-privilege users. This protection exists because kernel access grants extensive control over the entire system. Therefore, most operating systems limit regular users and applications to user space, where they can perform necessary tasks without gaining system-level control.

Kernel access enables users to modify the operating system's functionality, appearance, and behavior, but it also carries the risk of system crashes and complete failure. Even with these dangers, system administrators sometimes need careful kernel access for maintenance and security purposes.

▼ [12.1] WHAT IS A **KERNEL MODULE**?

The kernel functions as the operating system's command center, orchestrating all operations and mediating between user applications and hardware components like processors, memory, and storage devices. Linux uses a monolithic kernel design that supports dynamic module integration, allowing components to be added or removed as needed.

Kernel updates are sometimes necessary to incorporate new device drivers for graphics cards, Bluetooth adapters, or USB peripherals, along with filesystem drivers and system extensions. These components must integrate directly into the kernel for proper operation. While some systems require complete kernel rebuilding, compilation, and restart to add drivers, Linux offers the ability to load certain modules dynamically without this lengthy process.

These dynamic components, called loadable kernel modules (LKMs), require deep kernel access to function, which creates significant security vulnerabilities. Malicious software called rootkits exploits this necessity by infiltrating the kernel through LKMs. Once malware establishes itself at the kernel level, attackers can seize complete system control.

When hackers successfully convince Linux administrators to install compromised modules, they gain not only system access but also the ability to manipulate what the system reports about running processes, network ports, active services, storage capacity, and virtually all other system information. By tricking administrators into installing seemingly legitimate device drivers containing hidden rootkits, attackers can achieve total system and kernel dominance.

This exploitation method represents one of the most dangerous ways rootkits compromise Linux and other operating systems. Mastering LKM concepts is essential for both effective Linux administration and understanding sophisticated attack vectors.

▼ [12.2] CHECKING THE **KERNEL VERSION**

how to check your current system kernel and if it is working:

```
(jynx@kali)-[/usr/share/doc/rsyslog]
$ uname -a
Linux kali 6.12.33+kali-amd64 #1 SMP PREEMPT_DYNAMIC Kali 6.12.33-1kali1 (2025-06-25) x86_64 GNU/Linux
```

- `uname` = Unix name command
- `a` = "all" flag (shows all available information)

Alternatively, you can also check using:

```
(jynx@kali)-[/usr/share/doc/rsyslog]
$ cat /proc/version
Linux version 6.12.33+kali-amd64 (devel@kali.org) (x86_64-linux-gnu-gcc-14 (Debian 14.2.0-19) 14.2.0, GNU ld (GNU B
inutils for Debian) 2.44) #1 SMP PREEMPT_DYNAMIC Kali 6.12.33-1kali1 (2025-06-25)
```

What is /proc/version:

- A virtual file in the `/proc` filesystem (procfs)
- Contains kernel version information in a single line
- Automatically generated by the kernel
- Similar to `uname -a` but with additional compiler details

▼ [12.3] KERNEL TUNING WITH SYSCTL

Using appropriate commands allows you to customize your kernel by adjusting memory settings, activating network capabilities, and strengthening security against external threats. Contemporary Linux kernels rely on the `sysctl` utility for kernel parameter modification. Any adjustments made through `sysctl` are temporary and will revert when the system restarts. For persistent changes, you must directly modify the `sysctl` configuration file located at `/etc/sysctl.conf`.

Exercise

extreme caution when working with `sysctl`, as improper use without adequate knowledge and experience can render your system unable to boot or function properly. Always thoroughly evaluate your intended changes before implementing any **permanent modifications** to avoid system failure.

```
(jynx@kali)-[/usr/share/doc/rsyslog]
$ sysctl -a | head
sysctl: abi.vsyscall32 = 1
crypto.fips_enabled = 0
crypto.fips_name = Linux Kernel Cryptographic API
crypto.fips_version = 6.12.33+kali-amd64
debug.exception-trace = 1
debug.kprobes-optimization = 1
dev.hpet.max-user-freq = 64
dev.mac_hid.mouse_button2_keycode = 97
dev.mac_hid.mouse_button3_keycode = 100
dev.mac_hid.mouse_button_emulation = 0
permission denied on key 'kernel.apparmor_display_secid_mode'
sysctl: permission denied on key 'kernel.apparmor_restrict_unprivileged_unconfined'
```

To illustrate `sysctl` usage, consider enabling packet forwarding for man-in-the-middle (MITM) attacks. In MITM scenarios, attackers position themselves between communicating systems to capture transmitted data. Since traffic flows through the attacker's machine, they can monitor and potentially modify the communications. Packet forwarding activation is one method to establish this traffic routing.

When examining the output or filtering for IPv4 settings using `sysctl -a | less | grep ipv4.ip`, you'll find the relevant configuration line(s):

```
(jynx@kali)-[/usr/share/doc/rsyslog]
$ sysctl -a | less | grep .ipv4.ip
sysctl: permission denied on key 'kernel.apparmor_display_secid_mode'
sysctl: permission denied on key 'kernel.apparmor_restrict_unprivileged_unconfined'
sysctl: permission denied on key 'kernel.cad_pid'
sysctl: permission denied on key 'kernel.unprivileged_userns_apparmor_policy'
sysctl: permission denied on key 'kernel.usermodehelper.bset'
sysctl: permission denied on key 'kernel.usermodehelper.inheritable'
sysctl: permission denied on key 'net.core.bpf_jit_harden'
sysctl: permission denied on key 'net.core.bpf_jit_kallsyms'
sysctl: permission denied on key 'net.core.bpf_jit_limit'
net.ipv4.ip_autobind_reuse = 0
net.ipv4.ip_default_ttl = 64
net.ipv4.ip_dynaddr = 0
net.ipv4.ip_early_demux = 1
net.ipv4.ip_forward = 0
net.ipv4.ip_forward_update_priority = 1
net.ipv4.ip_forward_use_pmtu = 0
net.ipv4.ip_local_port_range = 32768 60999
net.ipv4.ip_local_reserved_ports =
net.ipv4.ip_no_pmtu_disc = 0
net.ipv4.ip_nonlocal_bind = 0
net.ipv4.ip_unprivileged_port_start = 0
net.ipv4.ipfrag_high_thresh = 4194304
net.ipv4.ipfrag_low_thresh = 3145728
net.ipv4.ipfrag_max_dist = 64
net.ipv4.ipfrag_secret_interval = 0
net.ipv4.ipfrag_time = 30
sysctl: permission denied on key 'net.ipv4.tcp_fastopen_key'
sysctl: permission denied on key 'net.ipv6.conf.all.stable_secret'
sysctl: permission denied on key 'net.ipv6.conf.default.stable_secret'
sysctl: permission denied on key 'net.ipv6.conf.eth0.stable_secret'
sysctl: permission denied on key 'net.ipv6.conf.lo.stable_secret'
sysctl: permission denied on key 'vm.mmap_rnd_bits'
sysctl: permission denied on key 'vm.mmap_rnd_compat_bits'
```

The parameter `net.ipv4.ip_forward = 0` controls the kernel's ability to forward received packets to their destinations - essentially acting as a relay that receives packets and retransmits them. The default value of 0 disables this forwarding capability. To activate IP forwarding, modify the value from 0 to 1 using this command:

```
sysctl -w net.ipv4.ip_forward=1
```

This change allows the system to route packets between network interfaces, enabling the MITM attack setup.

→ I'm not illustrating how to make the change permanent but I don't think it needs a Nicolas Tesla to figure that out themselves-

insert a wink emoji.

▼ [12.4] MANAGING KERNEL MODULES

Linux provides multiple methods for kernel module management. The traditional approach involves using commands from the `insmod` collection - where `insmod` means "insert module" and is designed specifically for module handling. An alternative method utilizes the `modprobe` command, which will be covered later in these series' chapters. In this section, we'll use the `lsmod`

command from the `insmod` toolkit to display the currently loaded kernel modules:

```
(jynx@kali)-[/usr/share/doc/rsyslog]
$ lsmod
Module                Size  Used by
cfg80211              1392640  0
snd_seq_dummy         12288  0
snd_seq_midi          20480  0
snd_hrtimer           12288  1
snd_seq_midi_event    16384  1 snd_seq_midi
snd_seq               110592  9 snd_seq_midi,snd_seq_midi_event,snd_seq_dummy
rfkill                40960  3 cfg80211
qrtr                  57344  2
sunrpc                872448  1
binfmt_misc           28672  1
snd_ens1371           36864  1
snd_ac97_codec        196608  1 snd_ens1371
ac97_bus              12288  1 snd_ac97_codec
intel_rapl_msr        20480  0
gameport              24576  1 snd_ens1371
snd_rawmidi           53248  2 snd_seq_midi,snd_ens1371
intel_rapl_common     53248  1 intel_rapl_msr
snd_seq_device        16384  3 snd_seq,snd_seq_midi,snd_rawmidi
snd_pcm               188416  2 snd_ac97_codec,snd_ens1371
```

The `lsmod` command displays all kernel modules along with details about their memory usage and interdependencies. For example, the `nfnetlink` module - a message-based communication protocol linking kernel and user space - occupies 16,384 bytes and serves both the `nfnetlink_log` and `nf_netlink_queue` modules.

Within the `insmod` toolkit, you can add modules using `insmod` and eliminate them with `rmmod` (remove module). However, these commands have limitations and don't properly handle module dependencies, potentially causing kernel instability or system failure. Due to these risks, current Linux distributions have incorporated the `modprobe` command, which automatically manages dependencies and provides safer module loading and removal operations. We'll examine `modprobe` shortly, but first let's explore how to gather additional module information.

To obtain detailed information about kernel modules, you can use the `modinfo` command. The command structure is simple: type `modinfo` followed by the specific module name you want to investigate. For instance, if you need basic details about the Bluetooth kernel module that appeared in your earlier `lsmod` output, you would use this command:


```
(jynx@kali)-[/usr/share/doc/rsyslog]
$ modinfo bluetooth
filename:      /lib/modules/6.12.33+kali-amd64/kernel/net/bluetooth/bluetooth.ko.xz
alias:         net-pf-31
license:       GPL
version:       2.22
description:    Bluetooth Core ver 2.22
author:        Marcel Holtmann <marcel@holtmann.org>
srcversion:    3E33401A497BCCFCDBAB5A2
depends:        rfkill,crc16
intree:        Y
name:          bluetooth
retpoline:     Y
vermagic:      6.12.33+kali-amd64 SMP preempt mod_unload modversions
sig_id:        PKCS#7
signer:        Build time autogenerated kernel key
sig_key:       73:62:2A:5E:3B:B2:A5:9C:3A:A4:5F:3A:24:79:39:F8:E3:35:49:6A
sig_hashalgo:  sha256
signature:     B3:C2:C9:D0:2B:D0:4A:94:83:F5:73:38:DE:49:EF:73:01:B7:7D:56:
               F9:0F:2A:16:76:49:94:17:29:5C:6A:2E:16:8F:0E:9C:FC:98:86:03:
               82:A3:D0:1D:9E:2F:DF:60:74:95:4C:2D:DB:3F:F6:E5:C8:5D:FB:03:
               51:D6:40:05:2B:FC:28:08:04:BF:84:52:0E:DD:AF:15:33:D7:A7:ED:
               93:80:EC:BC:CD:E6:EF:9F:05:DD:74:9A:53:6B:15:51:74:7A:5E:A5:
               8D:A0:B3:94:95:0A:06:4F:15:8D:F8:CA:61:D6:66:54:DB:F5:FE:EE:
               1F:74:7E:F9:E2:16:9C:AC:24:03:D7:1D:01:C9:2C:E9:E6:D0:C7:16:
               B4:A7:67:99:4C:4B:4B:37:85:A1:68:FA:35:07:9A:EE:86:F6:15:08:
               6E:73:94:62:9A:02:6F:79:72:C3:79:95:6D:06:CB:FE:16:6A:BC:42:
               ED:08:2C:C7:68:01:7C:62:1E:DB:6E:C3:6A:58:C9:A8:ED:33:F0:3E:
               E3:C1:6F:E3:39:BB:F8:04:68:E7:C9:7E:C9:41:A7:3F:59:3C:80:71:
               A3:22:3F:07:70:52:8C:5E:51:E3:38:4D:80:41:CA:24:DF:16:B4:A7:
               07:55:3D:6B:04:8E:35:4A:13:5F:D7:12:1D:F3:B8:6D:EF:06:F3:7F:
               7D:43:2E:05:09:9D:13:F3:6D:5B:83:84:B7:46:5F:DD:E2:ED:E8:97:
               04:DC:ED:BE:5E:F0:95:96:85:A5:47:CE:21:6F:3E:96:58:17:43:DA:
               7D:86:18:2C:54:49:B7:F4:FB:80:7D:B5:0D:A6:5B:CD:00:BF:32:BA:
               21:CE:09:3F:B2:73:76:9D:46:86:84:00:69:49:12:A2:C1:56:0B:AC:
               0C:CF:A6:B5:D3:78:59:D7:22:01:17:36:00:AA:C3:BF:B5:31:48:7D:
               0E:BE:C3:BC:C1:05:21:09:63:18:30:24:05:B7:1C:91:59:F0:57:40:
               06:32:8F:46:AC:30:BE:3E:E3:81:9C:4F:B9:44:96:AD:87:8A:2E:58:
               4D:C6:9E:8F:50:02:F4:84:A6:95:2C:48:D8:CE:DE:B2:36:E8:E2:ED:
               B0:39:25:6E:17:E8:A1:A0:B3:B3:10:62:30:3D:F3:C3:23:F4:21:66:
               10:53:89:E1:87:01:18:42:E2:E3:2A:B0:12:0E:B0:0F:A1:51:40:8A:
               81:5F:6B:DB:7C:0F:A6:57:04:BB:18:66:1F:14:79:C2:36:29:6D:02:
               86:4A:9F:80:C0:AA:4A:9D:3D:D4:0E:69:05:04:15:09:40:AF:9A:F6:
               02:B8:45:6C:49:24:92:3B:5F:88:58:E3
parm:          disable_esco:Disable eSCO connection creation (bool)
parm:          disable_ertm:Disable enhanced retransmission mode (bool)
parm:          enable_ecred:Enable enhanced credit flow control mode (bool)

(jynx@kali)-[/usr/share/doc/rsyslog]
$
```

▼ [12.5] ADDING and REMOVING KERNEL MODULES

Let's practice inserting and removing a sample module to become comfortable with this procedure. Suppose you've just installed a new graphics card and need to add its drivers. Keep in mind that device drivers typically integrate directly into the kernel to obtain necessary access for proper operation. This direct kernel access also creates opportunities for malicious actors to embed rootkits or surveillance tools.

For demonstration purposes (don't execute these actual commands), imagine we want to install a new graphics driver called `KillAllTheFiles`. You would add it to your kernel using:

```
| modprobe -a KillAllImageFiles
```

To verify successful module loading, run the `dmesg` command, which displays the kernel's message buffer, then filter for "image" to check for any error indicators:

```
| dmesg | grep image
```

Any kernel messages containing "video" will appear here. If no output displays, there are no messages with that term.

To remove the same module, use the identical command with the `-r` (remove) option:

```
| modprobe -r KillAllImageFiles
```

While loadable kernel modules provide convenience for Linux users and administrators, they represent a significant security vulnerability that professional hackers should understand. As mentioned previously, LKMs can serve as ideal delivery mechanisms for introducing rootkits into the kernel and causing system damage.